C++ PROGRAMMING TEXTBOOK NOTES

Module: Base Class Pointers & Dynamic Dispatch

Author: Gaurav Kandpal | **Platform:** CS-Simplified | **Level:** Beginner to Advanced

1. CORE POINTER MECHANICS IN INHERITANCE

The Base Class Pointer Principle:
In C++, a **Base Class Pointer (Parent*)** can legally store the memory address of a **Derived Class Object (Child)**. Because a Derived class 'Is-A' Base class, it inherits all members of the base class. However, the reverse is **strictly illegal**: a Derived class pointer cannot point to a Base class object without explicit, dangerous casting.

Key Mechanics & Limitations:

- **Base Pointer → Derived Object:** Fully permitted. Enables polymorphic interfaces.
- **Derived Pointer → Base Object:** Illegal at compile time (causes a type mismatch error).
- **Static Member Accessibility:** By default (without virtual), a Base pointer can **only access member functions and variables inherited from the Base class**.

```
// Base Class Pointer pointing to Derived Object (Static Binding)
#include <iostream>
using namespace std;

class Parent {
public:
    void show() {
        cout << "This is Parent Class show()\n";
    }
};

class Child : public Parent {
public:
    void show() {
        cout << "This is Child Class show()\n";
    }
};

int main() {
    Parent* ptr; // Base class pointer
    Child obj; // Derived class object
    ptr = &obj; // Base pointer holds Derived object address (VALID)

    ptr->show(); // CALLS Parent::show() due to Early/Static Binding!

    // Child* cptr = new Parent(); // ERROR: Cannot point Child pointer to Base object!
    return 0;
}
```

2. EARLY BINDING VS. LATE BINDING (VIRTUAL DISPATCH)

When executing ptr->show(); through a base pointer:
• **Without virtual (Early Binding):** The compiler binds the function call at **compile time** based strictly on the *pointer's declared type* (Parent*). Therefore, Parent::show() executes.
• **With virtual (Late / Dynamic Binding):** The compiler binds the function call at **runtime** based on the *actual object type in memory* (Child object). Therefore, Child::show() executes.

```cpp
// Enabling Dynamic Dispatch using the virtual Keyword
#include <iostream>
using namespace std;

class Parent {
public:
    virtual void show() { // virtual keyword instructs compiler to use Late Binding
        cout << "This is Parent Class show()\n";
    }
    virtual ~Parent() {} // Virtual destructor prevents memory leaks
};

class Child : public Parent {
public:
    void show() override { // Overrides Base virtual function
        cout << "This is Child Class show()\n";
    }
};

int main() {
    Parent* ptr = new Child(); // Base pointer points to heap-allocated Child
    ptr->show(); // OUTPUT: "This is Child Class show()"

    delete ptr; // Safely deallocates memory via virtual destructor
    return 0;
}
```

3. ARCHITECTURAL ANALOGY: UNIVERSAL CONTROLLER

REAL-WORLD ANALOGY: Universal Remote & Connected Home Appliances

- **Base Pointer (Universal Remote):** The reference variable standardizes the interface.
- **Derived Objects (Smart Devices):** Television, Air Conditioner, or Speaker units.
- **Virtual Function (Power Button):** Pressing *Power* triggers TV On on a TV instance, AC Cooling on an AC instance, and Audio Playback on a Speaker instance.
- **Why it Matters:** The client code operates entirely through the Base Remote interface without needing specific remote types for every device.

4. POINTER & BINDING MECHANICS COMPARISON

Property / Metric	Non-Virtual Base Pointer Call	Virtual Base Pointer Call
Binding Mechanism	**Early Binding** (Static / Compile-time).	**Late Binding** (Dynamic / Runtime).
Resolution Basis	Determined by **Pointer Type** (Parent*).	Determined by **Pointed Object Type** (Child).
Function Executed	Parent::show()	Child::show()
Internal Lookup	Direct offset / symbol resolution.	Indirection via **VPTR & VTABLE**.
Flexibility	Rigid; lacks dynamic behavior.	Polymorphic; foundation for clean OOP design.

5. SUMMARY & BEST PRACTICES

1. Always Use Virtual Destructors in Base Classes:

If you deallocate a derived object stored in a base pointer (e.g., delete ptr;), a non-virtual destructor in the Base class leads to **undefined behavior and memory leaks** as the Derived destructor won't be called.

2. Prefer C++ Standard Override Specifier:

Always mark overridden functions with override in derived classes so the compiler catches accidental parameter mismatch bugs instantly.